

UNIVERSIDADE FEDERAL DE SANTA MARIA (UFSM)
COLÉGIO POLITÉCNICO DA UFSM
CURSO TEC/SUP. TECNOLOGIA EM SISTEMAS PARA INTERNET

NATANAEL PASSINATO PIOVESAN

FLUTTER COM REATIVIDADE

Persistência em memória, Arquitetura em camadas, Provider e ChangeNotifier

SANTA MARIA
2026

NATANAEL PASSINATO PIOVESAN

FLUTTER COM REATIVIDADE

Persistência em memória, Arquitetura em camadas, Provider e ChangeNotifier

Relatório técnico apresentado ao Curso Tec/Sup.
Tecnologia em Sistemas para Inter-
net, como
parte dos requisitos necessários à obtenção do
título de Técnico em Informática para Internet.

Orientador: Fábio Parreira

SUMÁRIO

1 INTRODUÇÃO	3
2 OBJETIVO	3
3 ESTRUTURA	4
4 EXPLICAÇÃO DAS CAMADAS	4
5 ANÁLISE DA REATIVIDADE	5
6 FLUXO COMPLETO DA APLICAÇÃO	5
7 ÁRVORE DE WIDGETS	6
8 PRINTS DO APLICATIVO	7
9 TRECHOS DE CÓDIGO COMENTADOS	7
10 DIFICULDADES ENCONTRADAS	7
11 CONCLUSÃO	8
12 REFERÊNCIAS	8

1 INTRODUÇÃO

1.1 Tema

O projeto consiste no desenvolvimento de um sistema de gerenciamento de ativos de rede, focado especificamente na interface de gestão de switches. A aplicação atua como uma camada de software para o controle e documentação de infraestruturas de rede locais.

1.2 Problema resolvido

Em ambientes de TI, a organização física e lógica de switches costuma ser documentada de forma ineficiente, descentralizada (planilhas ou registros manuais) ou até mesmo inexistente. Isso dificulta a identificação rápida de portas ocupadas, dispositivos conectados e o rastreamento de falhas. O aplicativo resolve essa fragmentação ao centralizar a gestão dessas conexões em uma interface intuitiva.

1.3 Funcionalidades implementadas

O sistema automatiza o inventário de rede por meio de um CRUD funcional, permitindo o registro e a edição ágil de conexões. A solução utiliza indicadores visuais de status para sinalizar a integridade dos switches em tempo real, transformando dados técnicos em sinais intuitivos que otimizam a manutenção e a organização da infraestrutura.

1.4 Tecnologias utilizadas

O desenvolvimento utiliza o framework Flutter e a linguagem Dart para criar uma interface reativa e multiplataforma. A solução adota uma arquitetura em camadas para organizar a lógica de negócio, empregando o Provider com ChangeNotifier para o gerenciamento de estado e persistência em memória para o armazenamento ágil das conexões durante a execução do aplicativo.

2 OBJETIVO

O objetivo principal deste projeto é desenvolver uma aplicação móvel funcional utilizando o framework Flutter para a gestão de interfaces de switches, proporcionando uma ferramenta centralizada para o controle de conexões de rede. O trabalho busca aplicar de forma prática os conceitos de arquitetura em camadas,

segmentando o código entre UI (Page e Controller) e Data (Model, Repository e Service) para garantir a separação de responsabilidades e a manutenibilidade do sistema.

Além disso, o projeto visa implementar um fluxo de reatividade eficiente, utilizando o padrão Provider em conjunto com a classe ChangeNotifier para que a interface reflita mudanças de estado em tempo real durante as operações de cadastro, listagem, edição e exclusão (CRUD). Toda a manipulação de dados é realizada com persistência em memória, assegurando agilidade no processamento das informações durante o ciclo de execução do aplicativo.

3 ESTRUTURA

3.1 Árvore de diretórios

```
lib/  
├── main.dart  
├── UI/  
│   ├── Page/  
│   │   └── switch_page.dart  
│   └── Controller/  
│       └── switch_controller.dart  
└── Data/  
    ├── Model/  
    │   └── switch_model.dart  
    ├── Repository/  
    │   └── switch_repository.dart  
    └── Service/  
        └── switch_service.dart
```

3.2 Funções de cada pasta e arquivo

lib/ :diretório raiz com o código fonte do projeto;

main.dart :responsável por inicializar o app e configurar o Provider/MultiProvider;

UI/ :pasta utilizada para centralizar o que será visível ao usuário e a lógica de interação;

Page/ : Contém os Widgets de tela. Onde fica a interface gráfica do app;

Controller/ : Contém a lógica de estado da interface. Esta classe estende ChangeNotifier e é responsável por receber as ações do usuário e notificar a Page sobre mudanças nos dados.

Data/ : Pasta dedicada à gestão da informação e regras de negócio;

Model/ : Define a classe "Switch", especificando os atributos necessários (como IP, nome e status) e os métodos de conversão de dados.

Repository/ : Atua como um mediador. Sua função é intermediar o acesso entre o Controller e o Service, garantindo que o restante do app não precise saber de onde os dados vêm.

Service/ : Implementa o acesso real aos dados. Neste projeto, é onde ocorre a persistência em memória, armazenando a lista de conexões durante a execução do aplicativo.

4 EXPLICAÇÃO DAS CAMADAS

Model: Esta camada define a classe que representa a entidade principal do sistema, o Switch. Ela contém a estrutura de dados necessária para identificar cada dispositivo e pode incluir métodos de conversão para facilitar o transporte das informações.

Service: É a responsável direta por manipular o acesso aos dados. No contexto deste projeto, ela implementa a persistência em memória, mantendo a lista de conexões ativa durante a execução e executando as operações lógicas de armazenamento.

Repository: tua como um mediador ou ponte entre a camada de lógica visual (Controller) e a camada de dados (Service). Sua função é isolar a fonte dos dados, permitindo que o Controller interaja com o Repository sem precisar conhecer os detalhes técnicos de como as informações são guardadas no Service.

Controller: Esta camada gerencia o estado da aplicação e herda da classe ChangeNotifier. Ela processa as ações enviadas pela interface, solicita alterações ao Repository e, fundamentalmente, executa o método notifyListeners() sempre que ocorre uma mudança, disparando a atualização da tela.

Page: Representa a interface gráfica visível ao usuário. É composta por Widgets que exibem a listagem de switches e formulários de cadastro. Esta camada "escuta" o Controller para se reconstruir automaticamente assim que novos dados são processados.

5 ANÁLISE DA REATIVIDADE

Onde a reatividade é configurada no projeto?

Ela esta configurada no main.dart, sendo iniciada no método build de MyApp, onde o MaterialApp é envolvido por um MultiProvider e dentro dele o ChangeNotifierProvider<SwitchController> é criado para disponibilizar o controller para a árvore de widgets abaixo dele.

Qual classe permite o comportamento reativo?

A classe `SwitchController`, pois herda atributos da classe nativa `ChangeNotifier`, permitindo emitir sinais para a UI que os dados mudaram.

Onde o estado do projeto é armazenado?

Os dados reais ficam armazenados em memória na camada `Service`, na lista privada final `List<SwitchDevice> _connections = []`. Enquanto o estado reativo usado pela interface utiliza a lista `_connections` presente no `SwitchController`.

Quais widgets escutam mudanças no Controller?

A própria `build` do `SwitchPage` que está utilizando `context.watch<SwitchController>()`, assim a página inteira é usada como ouvinte e sempre que é detectada uma mudança de estado no `Controller`, a página é reconstruída com os dados atualizados.

Onde `notifyListeners()` é chamado?

Em `switch_controller.dart`, dentro do método `carregarConnections()`.

O que acontece quando `notifyListeners()` é executado?

O `SwitchController` emite um alerta para o `Provider`, que avisa os widgets ouvintes de que os dados foram atualizados, forçando o Flutter a executar novamente os métodos `build` destes widgets.

Qual é a diferença entre `context.watch` e `context.read`?

O `context.watch` opera como um ouvinte ativo, ouvindo ao `Controller` forçando o `rebuild` toda vez que `notifyListeners()` foi chamado, devendo ser utilizado apenas para elementos visuais. Enquanto `context.read` tem uma leitura mais pontual, acessando o `Controller` e disparando apenas uma função, mas não ouvindo futuras mudanças, devendo ser usado fora do ciclo de construção de telas.

Por que não se deve usar `context.watch` para executar ações como salvar, editar ou excluir?

Não faz sentido registrar a interface para escutar uma mudança de tela dentro de um evento que ocorre praticamente instantaneamente, pois são eventos isolados e pontuais.

Como ocorre o fluxo completo desde a ação do usuário até a atualização da interface?

Usuário toca no botão -> page chama o método do controller -> controller altera o estado -> controller aciona o `notifyListeners()` -> page ouve e é reconstruída automaticamente.

6 FLUXO COMPLETO DA APLICAÇÃO

Ação na interface: O usuário interage com o app clicando no botão “Salvar”.

Requisição ao Controller: Page utiliza o `context.read<SwitchController>()` para chamar o método no Controller.

Persistência de dados: O Controller intermediado pelo Repository solicita ao Service gravar um novo dispositivo.

Atualização do estado: Após a gravação, o Controller recupera a lista atualizada do repositório com o método `carregarConnections`.

Emissão de notificação: Por sequência o Controller executa o método `notifyListeners()`.

Aviso aos ouvintes: O `notifyListeners()` sinaliza ao Provider que o estado mudou. O Provider avisa todos os widgets que estão utilizando o `context.watch` (neste app é o `SwitchPage`).

Rebuild da tela: Logo ser sinalizado, o Flutter aciona novamente o método `build` da Page, atualizando a tela com os novos dados que foram inseridos.

7 ÁRVORE DE WIDGETS

7.1 Árvore simplificada

```
Scaffold
├── AppBar (Barra superior transparente com título e ícone de status)
├── Column (Corpo principal da aplicação)
│   ├── Row (Cabeçalho Dashboard)
│   │   ├── Text ("Portas Ativas")
│   │   └── Container (Fundo do contador)
│   │       └── Text (Contador numérico)
│   ├── Divider (Linha separadora)
│   └── Expanded (Ocupa o restante da tela)
│       ├── ListView.builder (Lista dinâmica de conexões)
│       │   ├── Card (Cartão de cada dispositivo)
│       │   │   ├── ListTile (Estruturação interna do cartão)
│       │   │   │   ├── Icon (Símbolo de rede - leading)
│       │   │   │   ├── Row (Título: LED de status + Nome do Switch)
│       │   │   │   ├── Text (Subtítulo: IP e MAC)
│       │   │   │   └── Row (Ações - trailing)
│       │   │       ├── IconButton (Ação de Editar)
│       │   │       └── IconButton (Ação de Excluir)
│       └── FloatingActionButton.extended (Botão flutuante principal inferior)
```

7.2 Classificação dos widgets

Widgets que exibem dados

Text: informações estáticas (ex: título "Painel de Controle") e dinâmicas resgatadas do estado (ex: device.nome).

Icon: símbolos para contexto visual da infraestrutura.

ListView.builder: lê a lista em memória no Controller e renderiza os itens na tela, criando os blocos visuais apenas quando são necessários.

Card e ListTile: Estruturam de forma amigável a exibição das propriedades do modelo (Nome, IP e MAC) de um switch específico, encapsulando os dados em um bloco organizado.

Container: Usado para espaçamento e forma visual, como o pequeno círculo verde que simula o LED de status ativo da porta de rede.

Widgets que executam ações

FloatingActionButton.extended: É o gatilho principal para executar a função `_abrirFormulario()`, mudando o contexto para a tela de inserção de dados.

IconButton: Escutam os toques rápidos do usuário na lista para disparar ações com parâmetros específicos (passando o ID/objeto do dispositivo para edição ou acionando o gatilho de exclusão).

TextField (dentro dos formulários): Capturam as entradas do teclado e aplicam a máscara de formatação correspondente (como a máscara de IP e MAC).

ElevatedButton e TextButton: Utilizados nos diálogos modais para confirmar (context.read) as ações de "Salvar/Atualizar/Excluir", ou para executar o descarte e fechamento da janela (Navigator.pop).

8 PRINTS DO APLICATIVO

Encontram-se em anexo aos demais arquivos da entrega final.

9 TRECHOS DE CÓDIGO COMENTADOS

Foram adicionados comentários explicativos nos arquivos e locais solicitados (main.dart, Controller, Repository, Service, Page, Provider, ChangeNotifier, notifyListeners(), context.watch e context.read)

10 DIFICULDADES ENCONTRADAS

Durante a codificação, enfrentei desafios específicos com a nomeação de classes. Ao utilizar o nome Switch para o modelo de dados, o compilador apontou conflitos diretos com palavras reservadas da linguagem Dart e com componentes visuais nativos da biblioteca do Flutter. Consegui resolver esse problema refatorando o código e renomeando as classes (passando a adotar SwitchDevice), atributos e arquivos do projeto, isolando o meu modelo das bibliotecas internas.

Outra dificuldade encontrada foi a assimilação da lógica de reatividade. Compreender o fluxo de atualização de estado e como conectar a interface gráfica aos controladores exige a consolidação de conceitos complexos. Para resolver isso, dediquei tempo extra na revisão dos materiais teóricos da disciplina e na leitura da documentação técnica, o que me permitiu aplicar o padrão Provider de forma funcional e segura.

11 CONCLUSÃO

O desenvolvimento deste projeto foi fundamental para consolidar meus conhecimentos práticos em Flutter e Dart. Criar o aplicativo de gestão de switches me permitiu entender, na prática, como aplicar conceitos importantes de arquitetura e reatividade. Ficou claro o quanto a arquitetura em camadas facilita o trabalho do desenvolvedor. Separar o que é visual (Page e Controller) do que é lógica e dados (Model, Repository e Service) deixou o código muito mais limpo, organizado e fácil de dar manutenção. Isso ajuda a evitar que uma simples alteração na interface quebre a forma como os dados são salvos.

Sobre a reatividade, aprender a usar o Provider junto com o ChangeNotifier foi o grande aprendizado do trabalho. Consegui entender o fluxo completo: como o método `notifyListeners()` avisa a tela para se atualizar sozinha e por que é essencial usar o `context.watch` apenas para desenhar as informações e o `context.read` para executar ações, como o clique dos botões.

No fim, o trabalho cumpriu seu objetivo. Além de entregar um aplicativo funcional e visualmente adequado, o projeto me deu uma base sólida de como estruturar um software de forma profissional, o que será essencial para os meus próximos passos na disciplina e área de desenvolvimento.

12 REFERÊNCIAS

FLUTTER. Flutter documentation. Google, 2026. Disponível em: <<https://docs.flutter.dev/>>. Acesso em: maio 2026.

GOOGLE. Gemini. Modelo de linguagem de inteligência artificial. 2026. Disponível em: <<https://gemini.google.com/>>. Acesso em: maio 2026.

PARREIRA, Fábio. Material de apoio e roteiro de atividade da disciplina de Programação para Dispositivos Móveis. Colégio Politécnico da UFSM (Ambiente Moodle). Santa Maria, 2026.